Program Flow:

Session	Topics Covered	Hands-on	Tools / Platforms	Duration
Day 1 - AI Agent and Agent Harness				
Session 1 **Agentic AI Foundations**	• Generative AI and agentic AI: producing a response versus completing a task • Anatomy of an agent: model, instructions, context, tools and the execution loop • Agent reasoning and the execution lifecycle • Tool calling: how a model requests a tool and how the result returns • An LLM application compared with an AI agent • Where an agent harness fits, and why one is needed	Build a basic agent: user request, LLM, tool, tool result, agent response. Trace one complete execution loop step by step.	Python LLM API (OpenAI or Claude)	**4 hrs** 1.5 hrs concept + 2.5 hrs hands-on
Session 2 **Building the Agent Harness**	• The harness as the engineering and control layer around the agent • Responsibilities of the harness versus the agent • Context manager, session state and input/output handling • Agent runtime and lifecycle management • Tool manager: registration and invocation • Error handling, retries and timeouts • Basic guardrails • Structured logging and execution tracing	Build a lightweight harness in plain Python around the Session 1 agent: context manager, agent runtime, tool manager, error handler and logger.	Python LLM API	**4 hrs** 1 hr concept + 3 hrs hands-on
			Day 1 subtotal	**8 hrs** 2.5 hrs concept + 5.5 hrs hands-on

Odoo ID: ----------

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

Session	Topics Covered	Hands-on	Tools / Platforms	Duration
Day 2 - Tool Integration and the Enterprise MCP Server				
Session 3 **Tool Contracts and MCP Fundamentals**	• What an agent tool is: definition, schema, input and output contracts • Function calling, tool responses and tool errors • Tool timeouts, retries and authorisation basics • Why MCP: the integration problem it addresses • MCP architecture: host, client and server • MCP tools, resources and prompts • Transports: stdio and Streamable HTTP • MCP lifecycle, tool discovery and invocation • MCP compared with direct API integration	Build three local tools (customer, vehicle and ticket information) and register them with the harness. Move the tools behind a first MCP server over stdio and inspect it with MCP Inspector.	Python MCP Python SDK MCP Inspector	**4 hrs** 1.5 hrs concept + 2.5 hrs hands-on
Session 4 **Building the Enterprise MCP Server**	• Structuring an enterprise MCP server • Tool definitions with typed input and output schemas • Integrating backend APIs and data sources • Resources for reference data • Error mapping and tool responses • Server-side logging • Moving from stdio to Streamable HTTP • MCP client inside the harness: discovery and invocation	Build an enterprise MCP server exposing get_customer_details, get_vehicle_details, search_service_information and create_support_ticket against simulated enterprise services. Connect the harness so the agent discovers and calls these tools over MCP.	MCP Python SDK (FastMCP) FastAPI SQLite Pydantic	**4 hrs** 1 hr concept + 3 hrs hands-on
			Day 2 subtotal	**8 hrs** 2.5 hrs concept + 5.5 hrs hands-on

Odoo ID: ----------

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

Session	Topics Covered	Hands-on	Tools / Platforms	Duration
Day 3 - Cloud Deployment and Cross-team Integration				
Session 5 **Enterprise Design and Cloud Deployment**	• Designing reusable tools and setting tool boundaries • Input and output schema design; API abstraction • Authentication and per-tool authorisation • Secrets management • Configuration for development and production environments • Containerising the MCP server • Deployment on a managed container service • Health checks and centralised logging • Basic security considerations	Containerise the MCP server and deploy it on the selected hyperscaler with secrets and authentication in place. Point the harness at the remote endpoint and call it over HTTPS.	Docker AWS, Azure or Google Cloud (one, per client) Cloud secrets store	**4 hrs** 1.5 hrs concept + 2.5 hrs hands-on
Session 6 **Cross-team Integration and Capstone**	• Patterns for sharing agentic systems: a shared MCP server, and an agent exposed as an MCP server • Tool contracts, naming and versioning • Discovery through an internal catalogue of MCP servers • Per-consumer credentials and rate limits • Operational ownership between producing and consuming teams	Capstone: each pod publishes its deployed system with a documented tool contract, then integrates another pod's system into its own agent. End-to-end demonstration, reviewed as the Formative Review.	Full reference stack from Sessions 1-5	**4 hrs** 1 hr concept + 1.5 hrs hands-on + 1.5 hrs Capstone review
			Day 3 subtotal	**8 hrs** 2.5 hrs concept + 4 hrs hands-on + 1.5 hrs Capstone review
			Total Duration	**24 hrs** 7.5 hrs concept + 15 hrs hands-on + 1.5 hrs Capstone review

Reference stack: Python, an LLM API (OpenAI or Claude), the official MCP Python SDK, FastAPI and SQLite for simulated enterprise services, Docker, and one hyperscaler's managed container service and secrets store. The harness is written in plain Python without an agent framework, so that every harness responsibility is visible in code participants have written themselves.

Odoo ID: ----------

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.